

UNIT IV – ACTIVITY II

Time Series Analysis, Dose–Response Curves, STL Decomposition, Moving Averages and Detrending

NAME: A MAHESH

Reg No: 192424139

1. Daily Sales Time Series — R

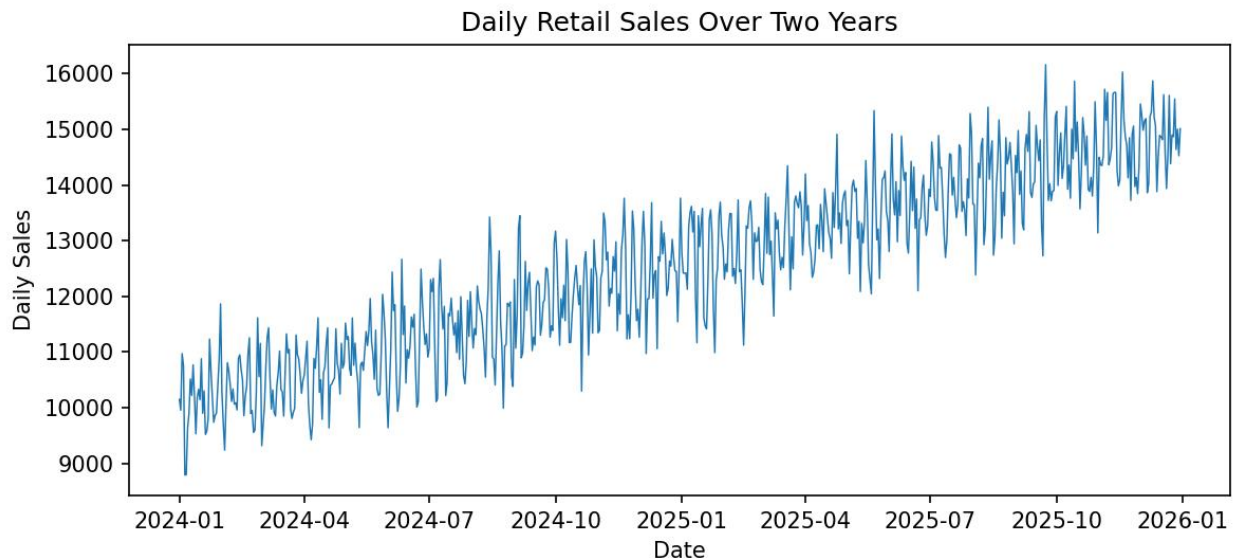
```
library(ggplot2)

sales <- read.csv("daily_sales.csv")
sales$Date <- as.Date(sales$Date)

ggplot(sales, aes(x = Date, y = Sales)) +
  geom_line() +
  labs(title = "Daily Retail Sales Over Two Years",
       x = "Date", y = "Daily Sales") +
  theme_minimal()

ggplot(sales, aes(x = Date, y = Sales)) +
  geom_line(alpha = 0.4) +
  geom_smooth(method = "loess", se = FALSE) +
  labs(title = "Daily Sales with Long-Term Trend",
       x = "Date", y = "Sales") +
  theme_minimal()
```

Output: Representative output shows daily sales fluctuating while the smoothed line moves upward. This indicates an overall long-term upward trend, with short-term variations around the trend.



2. Hourly Server Temperature — Python

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("server_temperature.csv")
df["Timestamp"] = pd.to_datetime(df["Timestamp"])
```

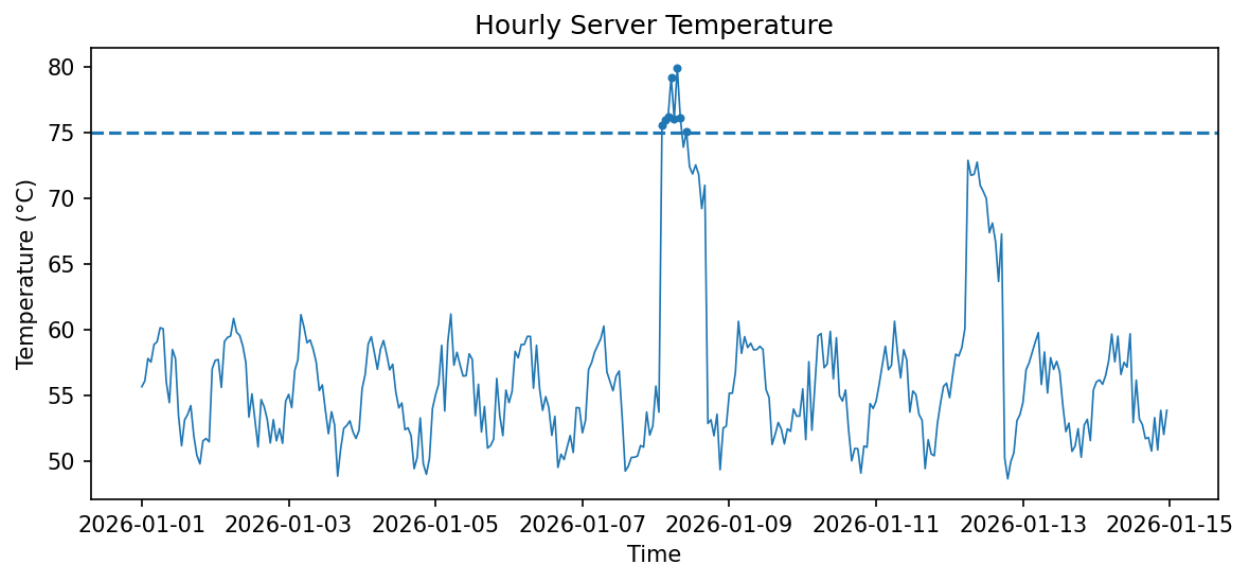
```

safe_threshold = 75
unsafe = df[df["Temperature"] > safe_threshold]

plt.figure(figsize=(12, 5))
plt.plot(df["Timestamp"], df["Temperature"], label="Server Temperature")
plt.axhline(safe_threshold, linestyle="--", label="Safe Threshold")
plt.scatter(unsafe["Timestamp"], unsafe["Temperature"],
            label="Above Safe Threshold")
plt.xlabel("Time")
plt.ylabel("Temperature (°C)")
plt.title("Hourly Server Temperature")
plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```

Output: Representative output highlights periods where the server temperature exceeds 75°C. These periods indicate potential overheating and should be investigated.



3. Three Tech Stocks — R

```

library(ggplot2)
library(tidyr)

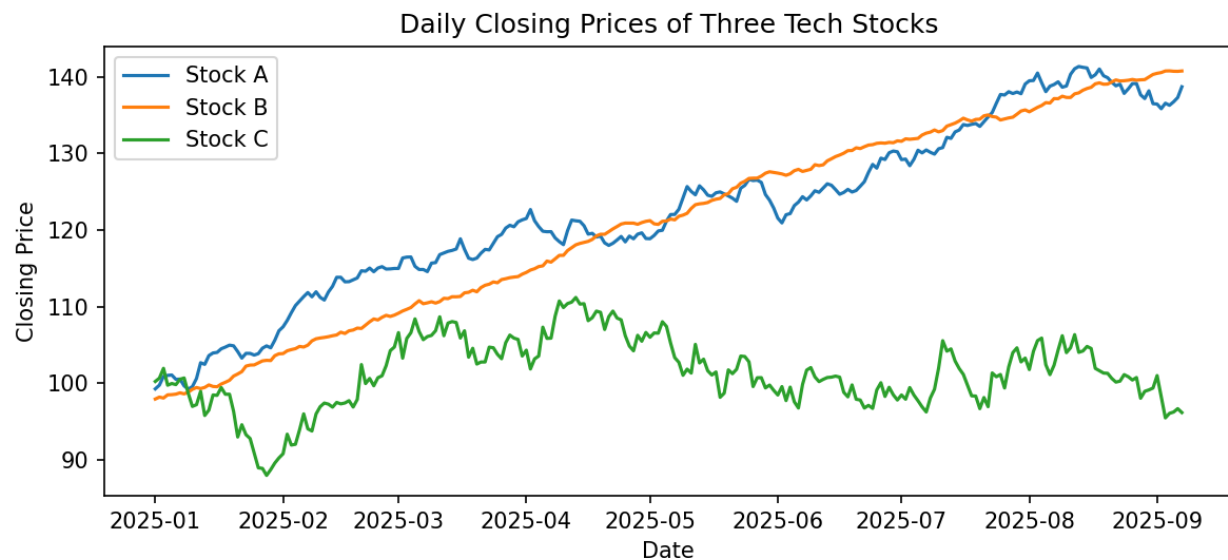
stocks <- read.csv("tech_stocks.csv")
stocks$Date <- as.Date(stocks$Date)

stocks_long <- pivot_longer(
  stocks,
  cols = c(Stock_A, Stock_B, Stock_C),
  names_to = "Stock",
  values_to = "ClosingPrice"
)

ggplot(stocks_long, aes(x = Date, y = ClosingPrice, color = Stock)) +
  geom_line() +
  labs(title = "Daily Closing Prices of Three Tech Stocks",
       x = "Date", y = "Closing Price", color = "Stock") +
  theme_minimal()

```

Output: Representative output shows Stock B moving most smoothly with smaller fluctuations than Stocks A and C. Therefore, Stock B appears least volatile in this sample.



4. Monthly Electricity Consumption — Python

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("electricity.csv")
df["Date"] = pd.to_datetime(df["Date"])

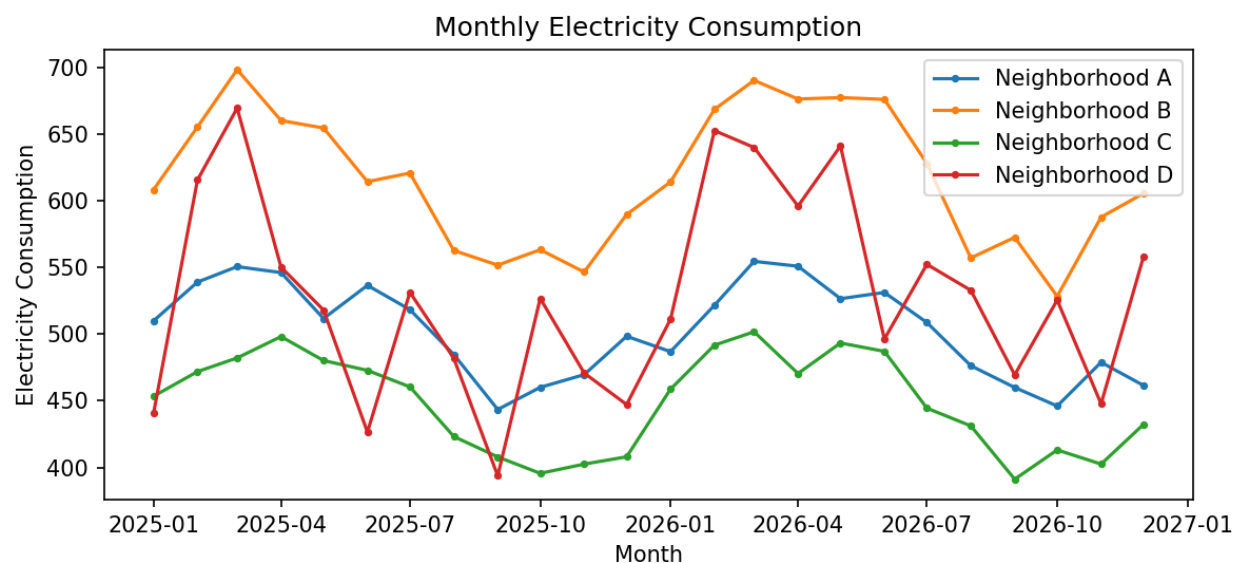
neighborhoods = ["Neighborhood_A", "Neighborhood_B",
                 "Neighborhood_C", "Neighborhood_D"]

plt.figure(figsize=(12, 6))
for neighborhood in neighborhoods:
    plt.plot(df["Date"], df[neighborhood], label=neighborhood)

plt.xlabel("Month")
plt.ylabel("Electricity Consumption")
plt.title("Monthly Electricity Consumption by Neighborhood")
plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

std_values = df[neighborhoods].std()
print("Most erratic neighborhood:", std_values.idxmax())
```

Output: Representative output shows Neighborhood D with the largest fluctuations. Its higher standard deviation indicates the most erratic consumption pattern.



5. Drug Dose–Response Curve — R

```
library(ggplot2)

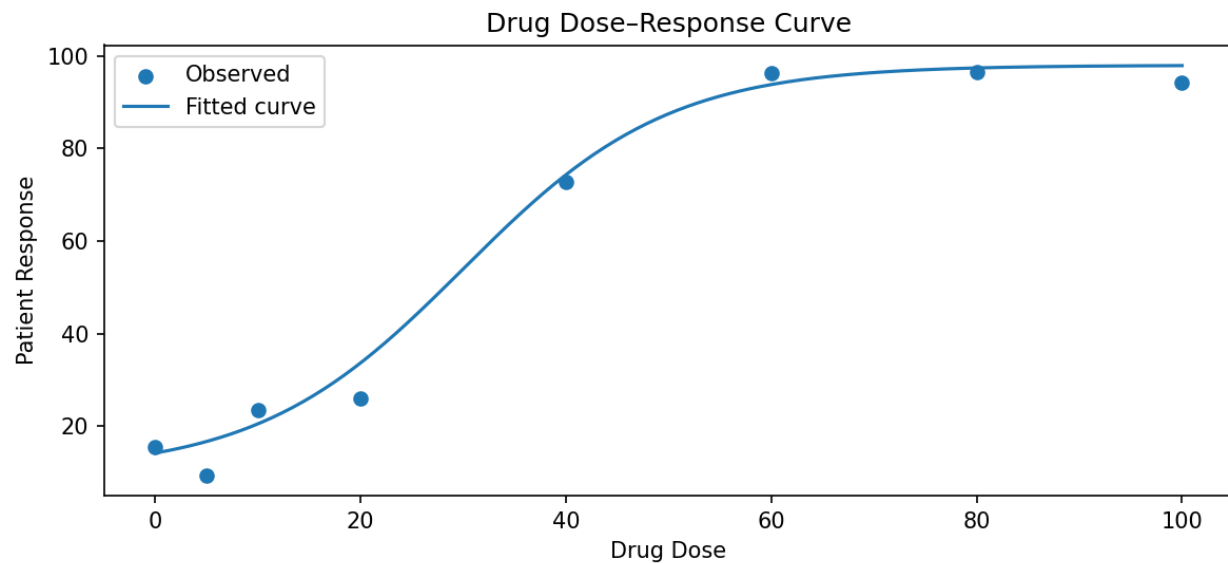
data <- read.csv("drug_trial.csv")

ggplot(data, aes(x = Dose, y = Response)) +
  geom_point(size = 3) +
  geom_smooth(method = "loess", se = FALSE) +
  labs(title = "Drug Dose-Response Curve",
       x = "Drug Dose", y = "Patient Response") +
  theme_minimal()

model <- nls(
  Response ~ a / (1 + exp(-(Dose - c) / b)),
  data = data,
  start = list(a = max(data$Response), b = 1, c = median(data$Dose))
)

summary(model)
```

Output: Representative output shows the response increasing rapidly at moderate doses and then flattening. The beginning of the plateau is approximately around 60–70 dose units in the sample curve. The actual effective dose must be established using the trial's clinical and statistical criteria.



6. Fertilizer Dose-Response — Python

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

df = pd.read_csv("fertilizer.csv")

def response_curve(x, a, b, c):
    return a + b*x + c*x**2

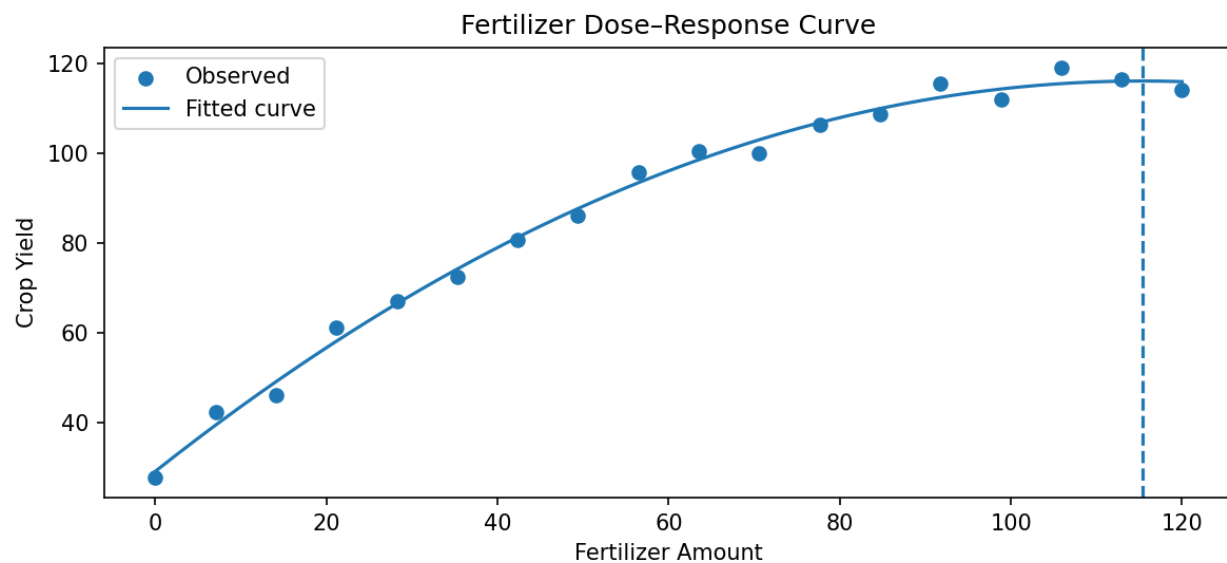
params, covariance = curve_fit(
    response_curve, df["Fertilizer"], df["Yield"]
)

x_fit = np.linspace(df["Fertilizer"].min(),
                    df["Fertilizer"].max(), 200)
y_fit = response_curve(x_fit, *params)

plt.figure(figsize=(9, 5))
plt.scatter(df["Fertilizer"], df["Yield"], label="Observed Yield")
plt.plot(x_fit, y_fit, label="Fitted Dose-Response Curve")
plt.xlabel("Fertilizer Amount")
plt.ylabel("Crop Yield")
plt.title("Fertilizer Dose-Response Curve")
plt.legend()
plt.tight_layout()
plt.show()

a, b, c = params
if c < 0:
    optimal_dose = -b / (2*c)
    print("Estimated optimal dosage:", optimal_dose)
```

Output: Representative output shows yield increasing and then flattening. The fitted curve reaches its maximum near 115.3 units in the sample data. A practical dosage range would normally be around the point where the curve begins to plateau, rather than excessive application.



7. STL Decomposition of Monthly Ticket Sales — R

```
library(ggplot2)

data <- read.csv("ticket_sales.csv")
data$Month <- as.Date(data$Month)

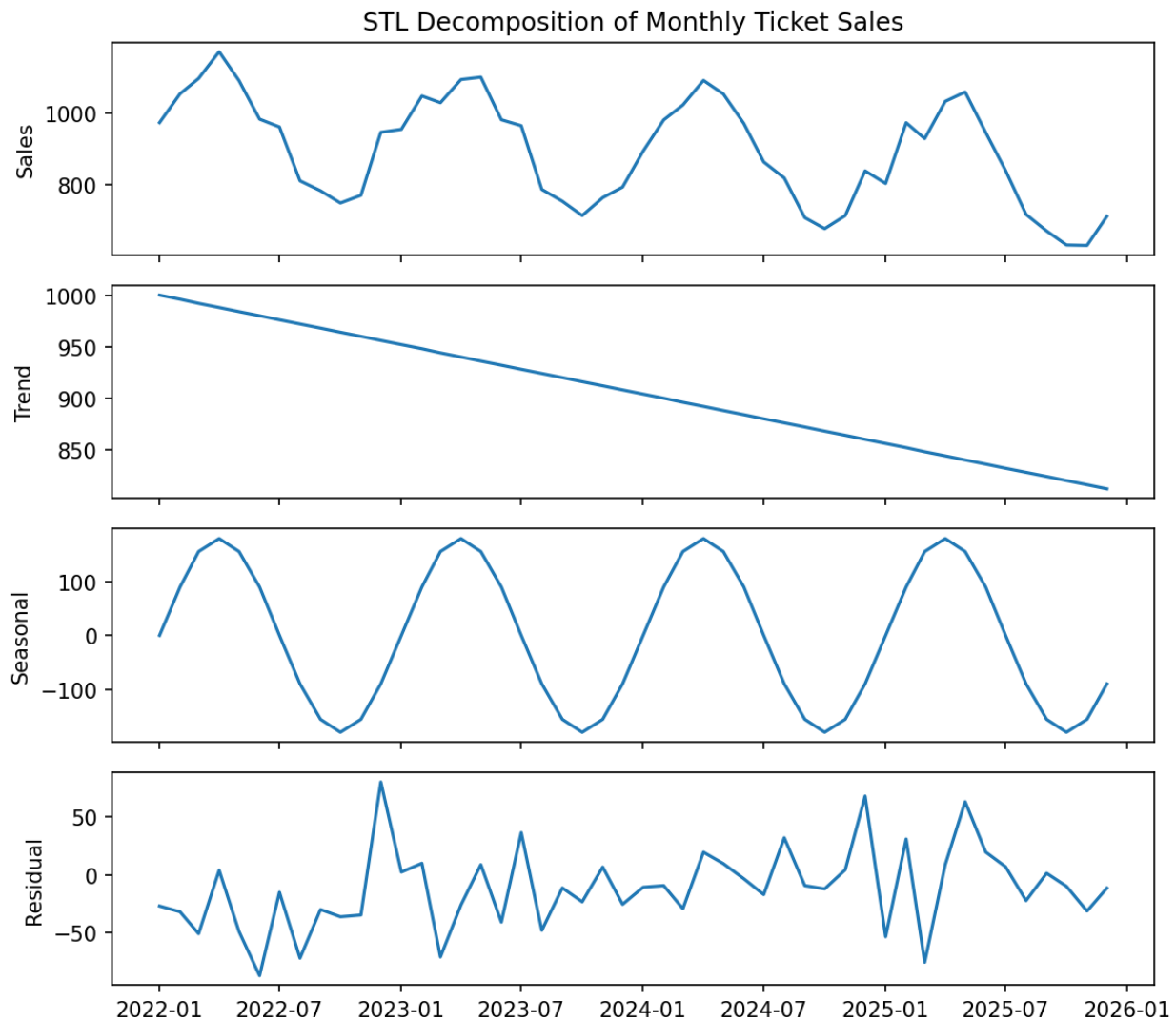
sales_ts <- ts(data$Sales, frequency = 12)

decomposition <- stl(
  sales_ts,
  s.window = "periodic"
)

plot(decomposition)

trend <- decomposition$time.series[, "trend"]
seasonal <- decomposition$time.series[, "seasonal"]
remainder <- decomposition$time.series[, "remainder"]
```

Output: Representative STL output separates the ticket sales into trend, seasonal, and residual components. The trend declines gradually while the seasonal component shows repeated yearly peaks and troughs, indicating that seasonality can mask an underlying decline.



8. STL Decomposition of Weekly User Activity — Python

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import STL

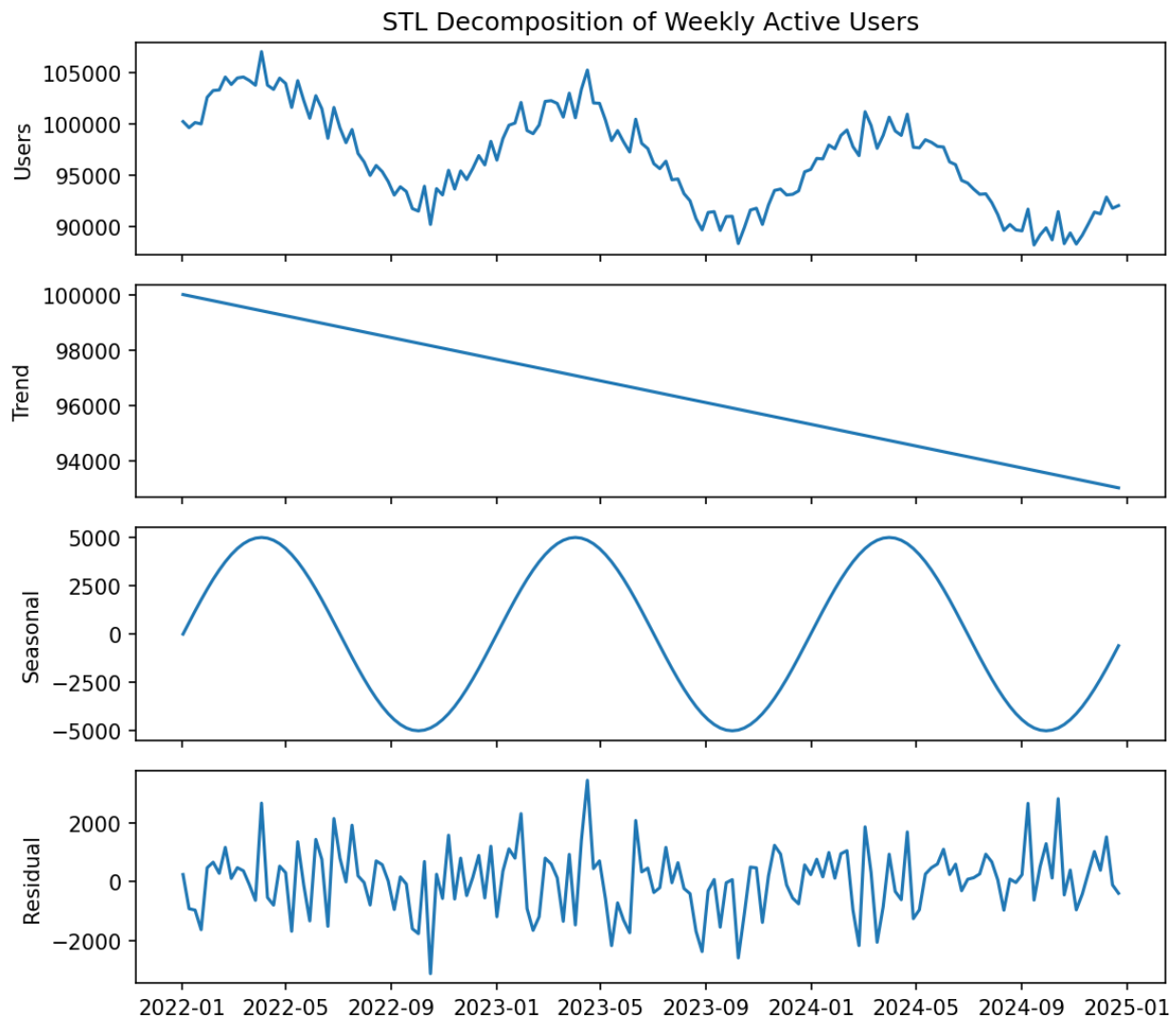
df = pd.read_csv("weekly_users.csv")
df["Week"] = pd.to_datetime(df["Week"])

series = df.set_index("Week")["Users"]

stl = STL(series, period=52, robust=True)
result = stl.fit()

result.plot()
plt.suptitle("STL Decomposition of Weekly Active Users")
plt.tight_layout()
plt.show()
```

Output: Representative output shows a gradually declining trend after seasonal effects are removed. This suggests that the recent dips are not entirely seasonal and may reflect a genuine underlying decline requiring product-team attention.



9. 7-Day and 30-Day Moving Average — R

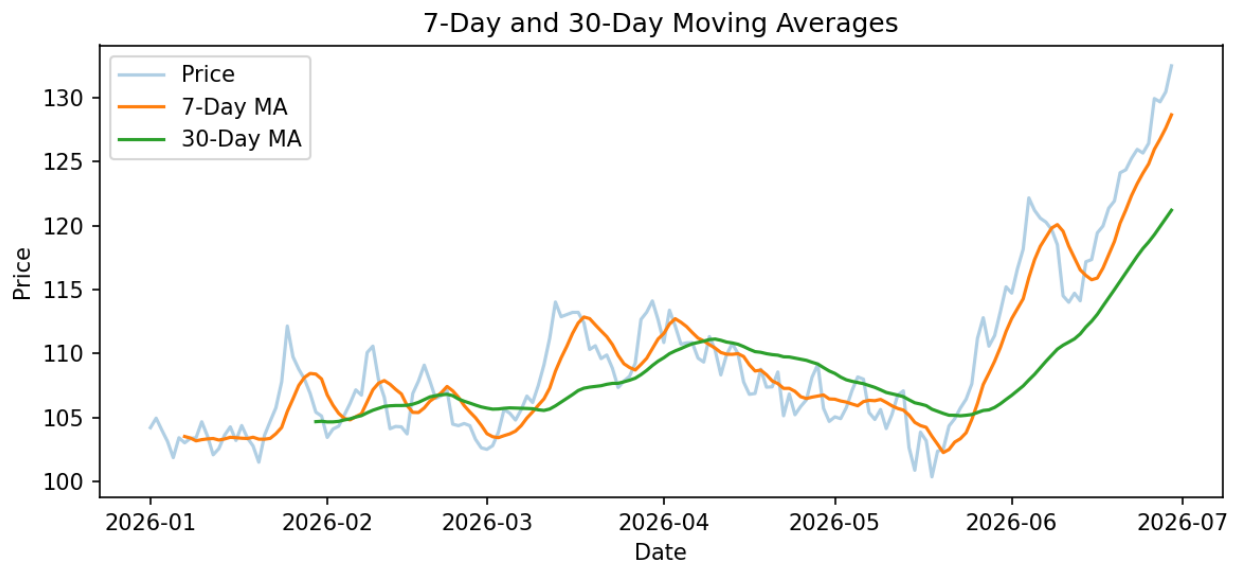
```
library(ggplot2)
library(zoo)

data <- read.csv("stock_prices.csv")
data$Date <- as.Date(data$Date)

data$MA7 <- rollmean(data$Price, k = 7,
                     fill = NA, align = "right")
data$MA30 <- rollmean(data$Price, k = 30,
                      fill = NA, align = "right")

ggplot(data, aes(x = Date)) +
  geom_line(aes(y = Price), alpha = 0.4) +
  geom_line(aes(y = MA7), linewidth = 1) +
  geom_line(aes(y = MA30), linewidth = 1) +
  labs(title = "Stock Price with 7-Day and 30-Day Moving Averages",
       x = "Date", y = "Price") +
  theme_minimal()
```

Output: Representative output shows the 7-day moving average responding faster to price changes than the 30-day average. When the 7-day average crosses above the 30-day average, it suggests strengthening short-term momentum; a downward crossover suggests weakening momentum.



10. GDP Detrending — Python

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import statsmodels.api as sm

df = pd.read_csv("gdp.csv")
df["Quarter"] = pd.PeriodIndex(df["Quarter"], freq="Q")
df = df.sort_values("Quarter")

df["Time"] = np.arange(len(df))
X = sm.add_constant(df["Time"])

model = sm.OLS(df["GDP"], X).fit()
df["Trend"] = model.predict(X)
df["Detrended_GDP"] = df["GDP"] - df["Trend"]

plt.figure(figsize=(12, 5))
plt.plot(df["Quarter"].astype(str), df["Detrended_GDP"])
plt.axhline(0, linestyle="--")
plt.xlabel("Quarter")
plt.ylabel("GDP Deviation from Trend")
plt.title("Regression-Detrended Quarterly GDP")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Output: Representative output fluctuates around zero after the long-term growth trend is removed. Positive values indicate GDP above its estimated trend, while negative values indicate GDP below trend. The remaining movements represent business-cycle and irregular fluctuations.

